

Lecture 05

Records, Variants, Pattern Matching

What is a Programming Language?

- **Syntax**: How to write programs down (spelling and grammar)
- **Semantics**: What programs mean (type checking and evaluation)
- **Idioms**: Typical ways to use language features for common needs
- **Libraries**: What is “standard” and easily available
- **Tools / Ecosystem**: What support do you get?
 - REPL, IDE, debugger, message board, governance, etc.
- **Community**: StackOverflow, open source projects, language cons, etc

What is a Programming Language?

- **Syntax**: How to
- **Semantics**: Wh
- **Idioms**: Typical
- **Libraries**: What
- **Tools / Ecosystem**
 - REPL, IDE,
- **Community**: Sta

We will mostly focus on **semantics** and **idioms**

- Carefully thinking about these makes us better programmers in *any* language
- Other parts of a PL are important and interesting
 - Especially when doing “real work”
 - ... but not our expertise 🧐

Building Types

- Already know many ways to build types
 - Base types : `bool` `int` `float` `string` ...
 - Type constructors : `t1 * t2` `t1 -> t2` `t list` `t option`
- Today: Define our own new types in OCaml
 - Especially enumerations and trees
 - Types often more important to program design than algorithms!
- But first: More general way to think about types in any language

Anatomy of types in ANY Language

Languages generally provide *three kinds of building blocks* to define types

- Two ways to “combine” existing types $t_1, t_2, \dots, t_N$ in a new type t
 - “Each Of” : a value of type t contains values of **each of** $t_1, \dots, t_N$
 - “One Of” : a value of type t contains values of **one of** $t_1, \dots, t_N$
- A way for values of type t to contain other (smaller) values of type t
 - “Self Reference” : a value of type t can contain other t values

The ability to nest and mix-and-match makes these building blocks even more powerful!



Composition



Example Types From Building Blocks

- Tuples are *each of*: an `int * bool` contains an `int` and a `bool`
- Options are *one of*: an `int option` contains an `int` or no data
- Lists throw in *self reference* to use all three building blocks:
 - An `int list` contains (an `int` and another `int list`) or no data
- Nesting lets us build all kinds of interesting types
 - `((int * bool) option * (string list list)) option list`

Example Types From Building Blocks

- Tuples are *each of*: an `int * bool` contains an `int` and a `bool`
- Options are *one of*: an `int option` contains an `int` or no data
- Lists throw in *self reference* to use all the things
 - An `int list` contains (an `int` and a `list`)
- Nesting lets us build all kinds of interesting types
 - `((int * bool) option * (string list list)) option list`

Types provide a compact language
for expressing the “shape” of data!

Today: Records and Variants

- **Records**: a new way to build each-of types, a little different than tuples
 - Named types
 - Named fields
- **Variants**: a new way to build our own one-of types
 - Named types
 - E.g., can define a type whose values hold an `int` or a `string`
 - To **build** variant values, type definitions include **constructors** like `Some`, `None`, `::`, `[]`
 - Do *not* think Java constructors
 - To **use** variant values, **match expressions** test and unpack them

Group Work

1. Define an OCaml type for a playing card
 - 4 suits: Hearts, Spades, Diamonds, Clubs
 - 13 values: 2-10, Jack, Queen, King, Ace
2. Write a function `is_face_card` that takes a card as input and returns `true` if the card is a face card, `false` otherwise
3. Write a function `is_flush` that takes a list of cards and returns `true` if the cards are all the same suit, `false` otherwise
 - Hint: You will need a helper function!



Records : Define

Syntax: `type tname = { f1 : t1; ... ; fN : tN }`

- `t1`, ..., `tN` are (already-defined) types
- `f1`, ..., `fN` are *field names* (same “spelling” rules as variables)

Adds new (record) type `tname` and fields `f1`, ..., `fN`

- Types are in their own name space -- types don't conflict with variables/functions, fields
- Field names are in their own name space -- fields don't conflict with variables/functions, types
- However, can *shadow* earlier type and field name definitions ⚠

Must define a record type before you can *build* or *use*

- That's just OCaml's rule (not fundamental)

Records : Build

Elements separated by semicolons “ ; ” ⚠

Get very weird error messages otherwise ⚠

Syntax: { $f_1 = e_1$; ... ; $f_N = e_N$ }

- e_1 , ..., e_N are expressions
- f_1 , ..., f_N are *field names*

Type Checking

- If t is defined as type $t = \{ f_1 : t_1; \dots ; f_N : t_N \}$ and
- e_1 has type t_1 and ... and e_N has type t_N
- Then { $f_1 = e_1$; ... ; $f_N = e_N$ } has type t
- Else, report error and fail

Records : Build

Syntax: $\{ f1 = e1; \dots ; fN = eN \}$

- $e1, \dots, eN$ are expressions
- $f1, \dots, fN$ are *field names*

Evaluation

- If $e1$ evaluates to value $v1$ and ... and eN evaluates to value vN
- Then $\{ f1 : e1; \dots ; fN : eN \}$ evaluates to
 $\{ f1 = v1; \dots ; fN = vN \}$



Records of values are values

Records : Use

Syntax: $e.f$

- Where e is an expression and f is a field name

Type Checking

- If e has type t and
- If t is defined as $\text{type } t = \{f_1:t_1; \dots; \underline{f:t_F}; \dots; f_N:t_N\}$
- Then $e.f$ has type t_F
- Else, report error and fail

Evaluation

- If e evaluates to $\{f_1=v_1; \dots; \underline{f=v_F}; \dots; f_N=v_N\}$
- Then $e.f$ evaluates to v_F

Tuples vs. Records

Same “expressive power” with
“ease of use tradeoffs”

- Semantically, not much difference. Compare:
 - `(1, true, "three")` versus `{a=1; b=true; c="three"}`
 - Tuples a little shorter, records easier to remember “what is where”
 - Avoid large tuples
 - Prefer records when multiple fields have the same type
- Common language design question:
By position (tuples) or by name (records)
 - Functions do some of each: *position* for caller but *name* for callee

self-documenting



Today: Records and Variants



- Records: a new way to build each-of types, a little different than tuples
 - Named types
 - Named fields
- Variants: a new way to build our own one-of types
 - Named types
 - E.g., can define a type whose values hold an `int` or a `string`
 - To **build** variant values, type definitions include **constructors** like `Some`, `None`, `::`, `[]`
 - Do *not* think Java constructors
 - To **use** variant values, **match expressions** test and unpack them

Today: Records and Variants



Records: a new way to build each

- Named types
- Named fields

● **Variants:** a new way to build out

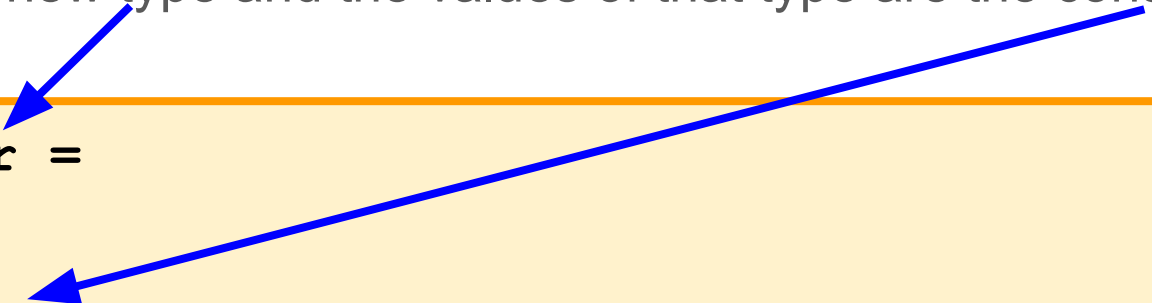
- Named types
- E.g., can define a type whose values hold an `int` or a `string`
- To **build** variant values, type definitions include **constructors** like `Some`, `None`, `::`, `[]`
 - Do *not* think Java constructors
- To **use** variant values, **match expressions** test and unpack them

Variants are not like records!

One-of types are not each-of types!

Enumerations

- Can use variant types to *enumerate* a fixed set of values for some new type
 - The simplest use of variant types, which are much more powerful
 - We start with this simplest use-case and build up quickly
- We name a new type and the values of that type are the constructors



```
type color =  
  | Red  
  | Green  
  | Blue
```

Variant definition syntax/semantics

```
type color = | Red | Green | Blue
```

- Syntax details:
 - First | optional
 - Constructors must start with capital
- Semantics: Adds to environment the type name and the constructors

Build

```
type color = | Red | Green | Blue
```

- These 3 constructors are now values (and therefore expressions)
 - 3 *different* ways to build a value of the new type **color**
 - There are no *other* values of the type **color**
 - E.g., [**Red**; **Blue**; **Blue**] has type **color list**

Bool

- One of the simplest enumerations is our old friend the booleans
- Could almost be defined `type bool = true | false`
- Only issue is capitalization rule (the language violated their own rule)
 - We can write `type mybool = True | False`

Anyway, we can now define enumerations and *build* values of them, so need a way to *use* values of them

Use variant types with match expressions

```
let string_of_color c =  
  match c with  
  | Red    -> "red"  
  | Green  -> "green"  
  | Blue   -> "blue"  
  
let s =  
  string_of_si_unit Green
```

- match expression with one branch for each constructor
- Like a multi-way if, with one branch for each constructor
- Guides type-checking:
 - all branches must have same type
 - *every constructor must have a branch*
- Guides evaluation rules:
 - evaluate exactly one branch

Back to bool

```
if e1 then e2 else e3
```

```
match e1 with  
| true  -> e2  
| false -> e3
```

- In fact, can use match expressions instead of conditional expressions
 - Yes, this works in OCaml
- No, *not* good style
 - *Demonstrates* that one feature (conditional expressions) can be explained entirely in terms of another more powerful feature (match expressions)
 - Another example of “**syntactic sugar**” (many more to come)

Beyond enumerations

As promised, OCaml variant types are much more powerful

- Each constructor can carry data [or not]
 - Type definition indicates the type of data each constructor carries
- Self-reference allowed in type definition
 - Can carry “another of the same thing”
- See code for three examples
 - A silly example to learn the syntax and semantics
 - An example for different kinds of shapes
 - An example that uses self-reference to define arithmetic-expression trees

Variants: Define

```
type silly =  
  | A of int * bool * string list  
  | Foo of string  
  | Pizza
```

Syntax: `type tname = | C1 [of t1] | ... | CN [of t1]`

- `t1`, ..., `tN` are types (and can include `t` inside them)
- `C1`, ..., `CN` are *constructors*
- Remember `[...]` is *metasyntax* for optional

Adds new (variant) type `tname` and constructors `C1`, ..., `CN`

- Types are in their own name space
- Constructors are in their own name space
- However, can *shadow* earlier type and constructor definitions ⚠

Must define a variant type before you can *build* or *use*

Variants: Build

```
type silly =  
  | A of int * bool * string list  
  | Foo of string  
  | Pizza
```

Syntax: $C [e]$

Type-checking:

- If C is in the environment due to type $t = \dots | C \text{ of } tC | \dots$
- And e has type tC (must provide correct arguments to C)
- Then $C \ e$ has type t
- Else does not type-check

Evaluation: Evaluate e to a value v and $C \ v$ is a value

- “Tags the data v with a C ”, producing a value of type t

Match expressions, the big picture

A powerful and concise expression form that combines three things:

1. Multiway conditional that branches on which variant of a one-of type you have
2. Extract the underlying data that was “tagged” with a constructor
3. Bind the extracted data to local variables that can then be used in the chosen conditional branch

This is elegant (once you get used to it) and avoids errors like forgetting cases or trying to extract the “wrong” data for the tag you have

Variants: Use

```
type silly =  
  | A of int * bool * string list  
  | Foo of string  
  | Pizza
```

[will generalize this soon!]

Syntax:

match *e* with | *P1* -> *e1* | *P2* -> *e2* ... | *PN* -> *eN*

- Where *e*, *e1*, ..., *eN* are expressions
- Where *P1*, ..., *PN* are patterns

For now, each pattern should be one of these:

- *C* if *C* is a constructor that carries no data
- *C* (*x1*, *x2*, ..., *xn*) if *C* is a constructor that carries an *n*-tuple of data
- *C* *x* otherwise

where *x1*, ..., *xn* and *x* are variables

Patterns

Patterns are not expressions, even though they kind of look like them

They are used for *matching* and to *bind local variables when they match*

Evaluation rules (again, will generalize soon)

match e with $P_1 \rightarrow e_1 \mid \dots \mid P_N \rightarrow e_N$

1. Evaluate e to some $C\ v$ (or just C)
2. Find the pattern P_i made with constructor C .
3. Create local bindings
 - C matching C creates no bindings
 - $C\ v$ matching $C\ x$ creates x bound to v
 - $C(v_1, \dots, v_n)$ matching $C(x_1, \dots, x_n)$ creates x_1 bound to v_1 , $\dots$, x_n bound to v_n
4. Evaluate the corresponding e_i using these local bindings to produce an overall result

Type-checking

Lots going on, toward two goals:

1. Ensure types work so the evaluation rules won't "run into problems"
2. Require one pattern for each constructor
 - Don't forget any
 - Don't repeat any

Type-checking

match e with $P1 \rightarrow e1 \mid \dots \mid PN \rightarrow eN$

1. Typecheck e to some type t where
type $t = \mid C1 \text{ [of } t1] \mid \dots \mid CN \text{ [of } t1]$
2. For each constructor $C1, \dots, CN$
 - a. Require exactly one pattern uses Ci
 - b. Require the pattern has “the right number of variables”
 - c. Require the corresponding ei type-checks in an environment where the variable(s) in the pattern have the type(s) indicated in the variant type definition
3. All ei need to have the same type and that is the overall type